



Hacking the Non-Windows Frontier



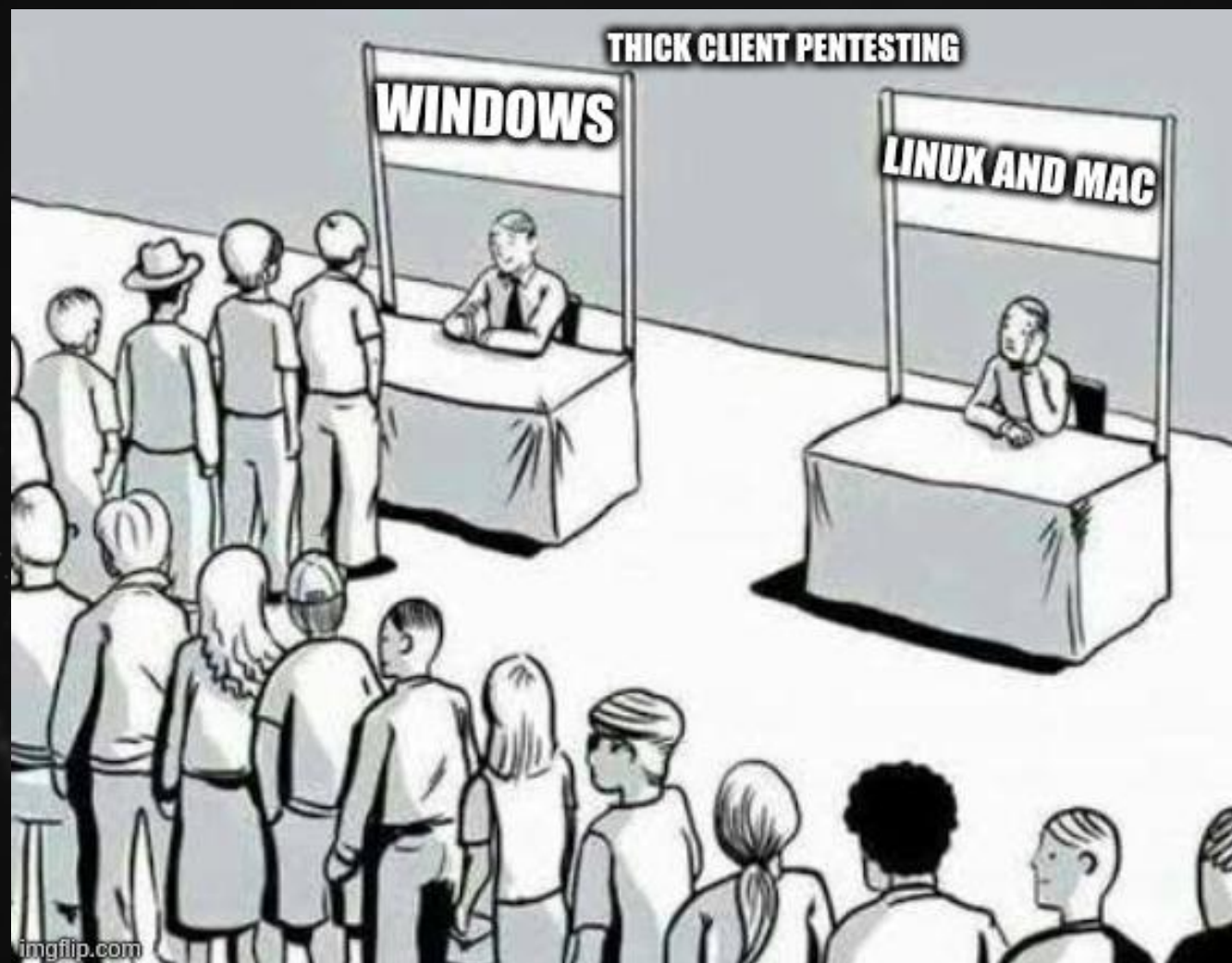
About the SPEAKER

Ajay SK

IoT Firmware Security Researcher, Payatu

- Part Time CTF player who loves to solve binary exploitation and reverse engineering challenges.
- Experienced in pentesting thick client applications on various platforms
- Green belt at pwn.college and secured multiple top positions in various CTF events.
- Nerd about learning Low level programming stuffs and memory corruption bugs.





SIT HERE HOOMAN

**LET'S TALK
ABOUT MAC FIRST**

imgflip.com

MacOS Versions(From SIP being introduced)

- [macOS 15 Sequoia](#), September 16, 2024
- [macOS 14 Sonoma](#), September 26, 2023
- [macOS 13 Ventura](#), October 25, 2022
- [macOS 12 Monterey](#), October 25, 2021
- [macOS 11 Big Sur](#), November 19, 2020
- [macOS 10.15 Catalina](#), October 7, 2019
- [macOS 10.14 Mojave](#), September 24, 2018
- [macOS 10.13 High Sierra](#), September 25, 2017
- [macOS 10.12 Sierra](#), September 20, 2016
- [OS X 10.11 El Capitan](#), September 30, 2015



System Integrity Protection

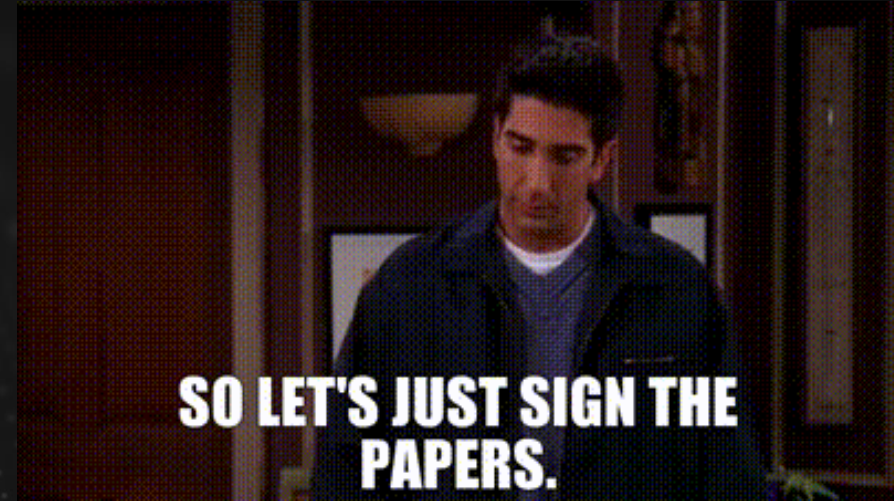
System Integrity Protection is a security technology designed to help prevent potentially malicious software from modifying protected files and folders on your Mac. System Integrity Protection restricts the root user account and limits the actions that the root user can perform on protected parts of the Mac operating system. Before System Integrity Protection (introduced in OS X El Capitan), the root user had no permission restrictions, so it could access any system folder or app on your Mac. Software obtained root-level access when you entered your administrator name and password to install the software. That allowed the software to modify or overwrite any system file or app.



Code Signing

Once an app is signed, the system can detect any change to the app—whether the change is introduced accidentally or by malicious code. You can control how your signed code loads signed plug-ins and other signed code without invalidating the signatures of the host code or of the guest (dynamically loaded) code.

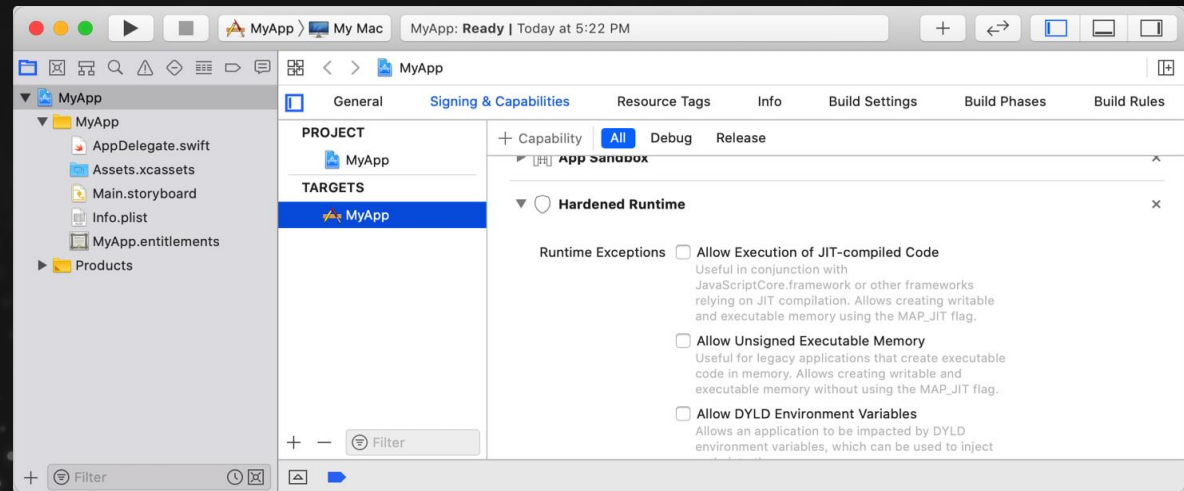
```
#define CS_RESTRICT          0x0000800    /*  
tell dyld to treat restricted */  
#define CS_REQUIRE_LV       0x0002000    /*  
require library validation */  
#define CS_RUNTIME          0x00010000   /*  
Apply hardened runtime policies */
```



Hardened Runtime

The Hardened Runtime, along with System Integrity Protection (SIP), protects the runtime integrity of your software by preventing certain classes of exploits, like code injection, dynamically linked library (DLL) hijacking, and process memory space tampering.

The Hardened Runtime doesn't affect the operation of most apps, but it does disallow certain less common capabilities, like just-in-time (JIT) compilation. If your app relies on a capability that the Hardened Runtime restricts, add an entitlement to disable an individual protection.



Hardened Runtime, Code Signing and SIP



Yeah I get it peeps, many security mitigations are being present on this platform which will make our attack surface limited and all. But what if I say, there is a lot more we can do that from application logic side that can yield high impactful vulnerabilities



ICANHASCHEEZBURGER.COM

Dylib Injection(DLL injection but on MacOS)

- DYLD_INSERT_LIBRARIES environment variable
- LC_LOAD_WEAK_DYLIB (traditional method)
- Injecting dylib into controlled rpath
- Abusing Access Control Logic for proxying dylibs



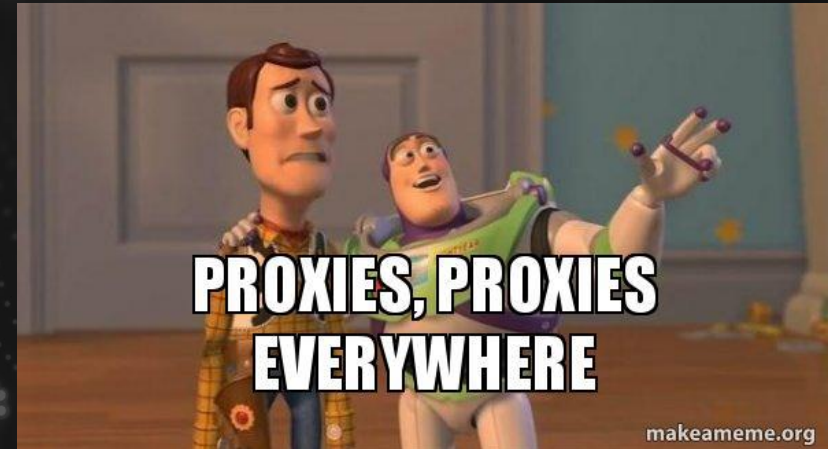
Injecting dylib into rpath

- The **LC_LOAD_DYLIB** header in a Mach-O binary tells macOS and OS X which dynamic libraries (dylibs) to load during execution time, if the dylib is not present the process will terminate.
- Copying the app to the folder to your control and abuse the rpath by placing the dylib to get a Code injection (But no lpe's would be there as it is will be now run as user perms and Code Signing Flags still cannot be bypassed)
- The binary with TCC entitlements are still affected by this technique alone. As DYLD_INSERT_LIBRARIES will be removed but the dylib injected into rpath will give us the entitlements.
- AMFI Launch Constraints are created in MacOS that stops the above attack but a developer has to setup it up
- <https://blog.xpnsec.com/bypassing-macos-privacy-controls/>



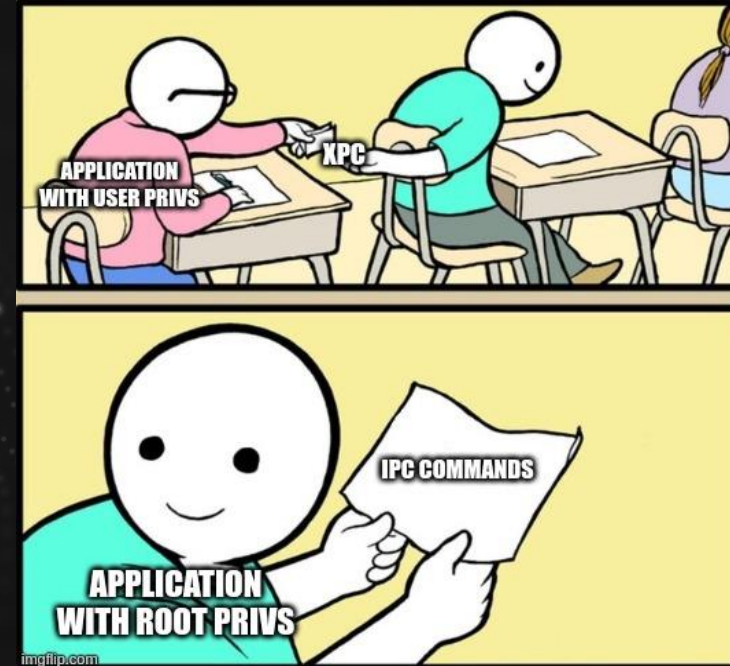
Abusing Access Control logic to proxying dylibs

- Abusing write access into the rpath dylibs to replace the original dylib with our malicious dylib and proxy with the original dylib.
- Abusing the /Applications directory ownership of admin group even without sudo access we can create the same directory structure for the Mac Store Apps and other Applications
- Install those applications which will be done by installd as root and wont replace the original permissions . So now we have write access to directory access even in those directory which we don't have normally.(Dylib injection time)



Privileged Helper Tools(XPC)

- Making Unauthenticated IPC Calls to the Privileged Applications if checks are not Implemented.
- Abusing .dylib injection to make IPC calls as a legitimate client
- PID Reuse attack
- Incorrect Client Verification to bypass security and make profits.



Pid Reuse Attack

- MacOS pids can be reused by passing the flag `POSIX_SPAWN_SETEXEC` | `POSIX_SPAWN_START_SUSPENDED` to `posix_spawn`. So, by this we can fork a process send IPC call to the privileged applications and then `posix_spawn` the legitimate client and if checks are made through pid number of the process, the checks are easily bypassed, and our Unauthenticated IPC call is accepted to achieve something crucial.



Incorrect Client Verification to bypass security and make profits.

- If the application with root privs checks for the code signing and bundle ID's.
- Some application are called by multiple client, So it has multiple bundle ID identifiers for check like MSAU(Microsoft Autoupdate). If one of the client,old versions can be run, and it suffers from code injection . We can use that as a springboard to attack the IPC resulting in a privileged action.



Local Privilege Escalation

- Abusing PATH variable writable location(/usr/local/bin) until Ventura(<https://abrictosecurity.com/privilege-escalation-on-macos-leveraging-common-techniques/>)
- Overwriting scripts and binaries that will be executed by root process by abusing access control logic
- Abusing dylib injection on binaries that are run as root
- Abusing XPC to achieve Local Privilege Escalation or atleast gain Privileged Actions (<https://www.offsec.com/blog/microsoft-teams-macos-local-privesc/>)
- Atleast Arbitrary File Deletion from low privileges(Privileged Actions)eg:-Directory Symlink Attack



Let's talk about Linux

- By default, the directory access control for normal user accounts and user with sudo privs are not so different. So no ways like mac to abuse directory perms to get a write into application folder directories.
- By default, the PATH default directory for root process can't be tampered from low privs process.
- Setuid binaries wont run with LD_PRELOAD libraries even without any special segments.



Shared Object Injection(.so file injection)

- Linux doesn't have the capability of mac to perform the code signing . So,no ways to check for shared object injection abuse.
- LD_PRELOAD variable to inject your .so files into the running process.
- Manipulating rpath entries to drop your .so files
- A lot of things can be done and prevention is harder



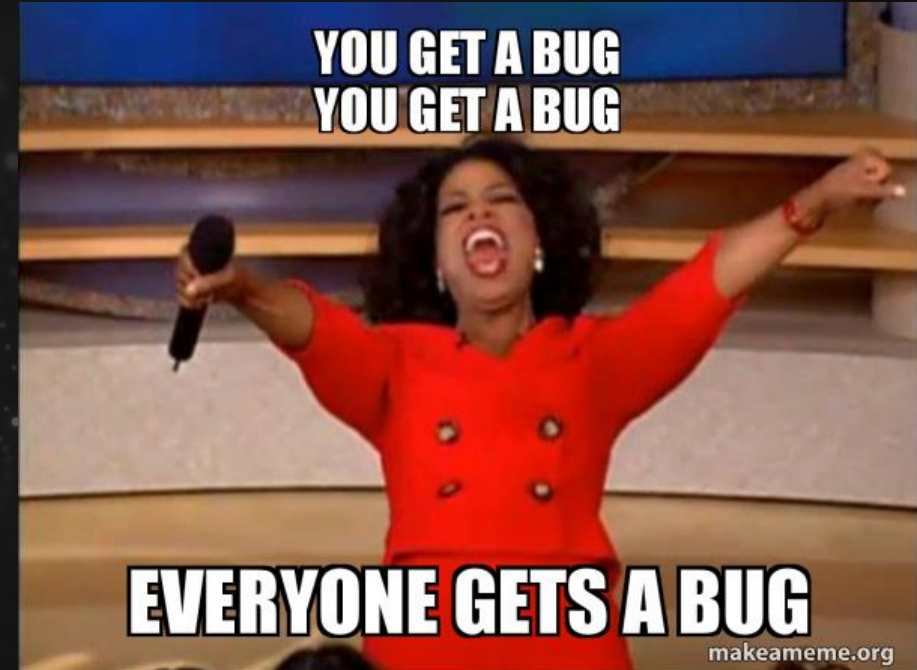
Privileged Helper Tools(DBUS)

- Making unauthenticated IPC calls to the application that without any checks performs the privileged actions
- Abusing the IPC Component of the root privs application by making authentic IPC calls originating from the user application.
- Sometimes poisoned packageInstaller IPC calls may lead to LPE or else atleast Privileged File Writes or File deletion.



Last but not the least

- Abusing Directory Control issues to read application sensitive world readable files.
- Ability to apply config to the application that might yield a primitive.
- Targeting arbitrary file deletion or Arbitrary file writes with high privileges from application logic to cause havoc on the system.
- Hunt for memory corruptions in the application (Low Privileged control file parsed by the high privs applications or send crafted packets)





Thank you



Contact me on



https://www.instagram.com/itz_me_strikerr__



<https://www.linkedin.com/in/ajay-sk-33ab591b0/>



<https://discordapp.com/users/640440268763430922>



Payatu

Securing the Future

REACH US



www.payatu.io



info@payatu.io

FOLLOW US

